Trabajo Práctico - SQL

A) Base de datos relacional

Consultas simples (una sola tabla)

1) Obtener una lista con los nombres de las distintas oficinas de la empresa.

```
SELECT DISTINCT descripcion FROM Oficinas;
```

2) Obtener una lista de todos los productos indicando descripción del producto, su precio de costo y su precio de costo IVA incluído (tomar el IVA como 21%).

```
SELECT descripcion, precio, precio * 1.21 FROM Productos;
```

3) Obtener una lista indicando para cada empleado apellido, nombre, fecha de cumpleaños y edad.

```
SELECT apellido, nombre, fecha_nacimiento, YEAR(CURDATE()) - YEAR(fecha_nacimiento) AS edad
FROM Empleados;
```

4) Listar todos los empleados que tiene un jefe asignado.

```
SELECT * FROM Empleados WHERE cod_jefe IS NOT NULL;
```

5) Listar los empleados de nombre “María” ordenado por apellido.

```
SELECT * FROM Empleados WHERE nombre = 'María' ORDER BY apellido;
```

6) Listar los clientes cuya razón social comience con “L” ordenado por código de cliente.

```
SELECT * FROM Clientes WHERE razón_social LIKE 'L%' ORDER BY cod_cliente;
```

7) Listar toda la información de los pedidos de Marzo ordenado por fecha de pedido.

```
SELECT * FROM Pedidos WHERE MONTH(fecha_pedido) = 3 ORDER BY fecha_pedido;
```

8) Listar las oficinas que no tienen asignado director.

```
SELECT * FROM Oficinas WHERE codigo_director IS NULL;
```

9) Listar los 4 productos de menor precio de costo.

```
SELECT * FROM Productos ORDER BY precio LIMIT 4;
```

10) Listar los códigos de empleados de los tres que tengan la mayor cuota.

```
SELECT cod_empleado FROM Datos_contratos ORDER BY cuota DESC LIMIT 3;
```

Consultas multitablas

1) De cada producto listar descripción, razón social del fabricante y stock ordenado por razón social y descripción.

```
SELECT Productos.descripcion, Fabricantes.razón_social, Productos.cantidad_stock
FROM Productos, Fabricantes
WHERE Productos.cod_fabricante = Fabricantes.cod_fabricante
ORDER BY Fabricantes.razón_social, Productos.descripcion;
```

2) De cada pedido listar código de pedido, fecha de pedido, apellido del empleado y razón social del cliente.

```
SELECT Pedidos.cod_pedido, Pedidos.fecha_pedido, Empleados.apellido, Clientes.razón_social
FROM Pedidos, Empleados, Clientes
WHERE Pedidos.cod_empleado = Empleados.cod_empleado
  AND Pedidos.cod_cliente = Clientes.cod_cliente;
```

3) Listar por cada empleado apellido, cuota asignada, oficina a la que pertenece ordenado en forma descendente por cuota.

```
SELECT Empleados.apellido, Datos_contratos.cuota, Oficinas.descripcion
FROM Empleados, Datos_contratos, Oficinas
WHERE Empleados.cod_empleado = Datos_contratos.cod_empleado
  AND Empleados.cod_oficina = Oficinas.cod_oficina
ORDER BY Datos_contratos.cuota DESC;
```

4) Listar sin repetir la razón social de todos aquellos clientes que hicieron pedidos en Abril.

```
SELECT DISTINCT Clientes.razón_social
FROM Clientes, Pedidos
WHERE Clientes.cod_cliente = Pedidos.cod_cliente
  AND MONTH(Pedidos.fecha_pedido) = 4;
```

5) Listar sin repetir los productos que fueron pedidos en Marzo.

```
SELECT DISTINCT Productos.descripcion
FROM Productos, Detalle_pedidos, Pedidos
WHERE Productos.cod_producto = Detalle_pedidos.cod_producto
  AND Detalle_pedidos.cod_pedido = Pedidos.cod_pedido
  AND MONTH(Pedidos.fecha_pedido) = 3;
```

6) Listar aquellos empleados que están contratados por más de 10 años ordenado por cantidad de años en forma descendente.

```
SELECT Empleados.apellido, Empleados.nombre,
       YEAR(CURDATE()) - YEAR(Datos_contratos.fecha_contrato) AS anios
FROM Empleados, Datos_contratos
WHERE Empleados.cod_empleado = Datos_contratos.cod_empleado
  AND YEAR(CURDATE()) - YEAR(Datos_contratos.fecha_contrato) > 10
ORDER BY anios DESC;
```

7) Obtener una lista de los clientes mayoristas ordenada por razón social.

```
SELECT Clientes.*
FROM Clientes, Listas
WHERE Clientes.cod_lista = Listas.cod_lista
  AND Listas.descripción = 'Mayorista'
ORDER BY Clientes.razón_social;
```

8) Obtener una lista sin repetir que indique qué productos compró cada cliente, ordenada por razón social y descripción.

```
SELECT DISTINCT Clientes.razón_social, Productos.descripcion
FROM Clientes, Pedidos, Detalle_pedidos, Productos
WHERE Clientes.cod_cliente = Pedidos.cod_cliente
  AND Pedidos.cod_pedido = Detalle_pedidos.cod_pedido
  AND Detalle_pedidos.cod_producto = Productos.cod_producto
ORDER BY Clientes.razón_social, Productos.descripcion;
```

9) Obtener una lista con la descripción de aquellos productos cuyo stock está por debajo del punto de reposición indicando cantidad a comprar y razón social del fabricante ordenada por razón social y descripción.

```
SELECT Productos.descripcion,
       Productos.punto_reposición - Productos.cantidad_stock AS cantidad_a_comprar,
       Fabricantes.razón_social
FROM Productos, Fabricantes
WHERE Productos.cod_fabricante = Fabricantes.cod_fabricante
  AND Productos.cantidad_stock < Productos.punto_reposición
ORDER BY Fabricantes.razón_social, Productos.descripcion;
```

10) Listar aquellos empleados cuya cuota es menor a 50000 o mayor a 100000.

```
SELECT Empleados.*, Datos_contratos.cuota
FROM Empleados, Datos_contratos
WHERE Empleados.cod_empleado = Datos_contratos.cod_empleado
  AND (Datos_contratos.cuota < 50000 OR Datos_contratos.cuota > 100000);
```

B) Tablas CLIENTES, PRODUCTOS, ITEM_VENTAS y VENTAS

1) Obtener la cantidad de unidades máxima.

```
SELECT MAX(cantidad) FROM ITEM_VENTAS;
```

2) Obtener la cantidad total de unidades vendidas del producto c.

```
SELECT SUM(cantidad) FROM ITEM_VENTAS WHERE codigo_producto = 1;
```

3) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado de mayor a menor por las cantidades vendidas.

```
SELECT PRODUCTOS.nombre_producto, SUM(ITEM_VENTAS.cantidad) AS total
FROM PRODUCTOS, ITEM_VENTAS
WHERE PRODUCTOS.codigo_producto = ITEM_VENTAS.codigo_producto
GROUP BY PRODUCTOS.nombre_producto
ORDER BY total DESC;
```

4) Cantidad de unidades vendidas por producto, indicando la descripción del producto, ordenado alfabéticamente por nombre de producto para los productos que vendieron mas de 30 unidades.

```
SELECT PRODUCTOS.nombre_producto, SUM(ITEM_VENTAS.cantidad) AS total
FROM PRODUCTOS, ITEM_VENTAS
WHERE PRODUCTOS.codigo_producto = ITEM_VENTAS.codigo_producto
GROUP BY PRODUCTOS.nombre_producto
HAVING SUM(ITEM_VENTAS.cantidad) > 30
ORDER BY PRODUCTOS.nombre_producto;
```

5) Obtener cuantas compras (1 factura = 1 compra) realizó cada cliente indicando el código y nombre del cliente ordenado de mayor a menor.

```
SELECT CLIENTES.codigo_cliente, CLIENTES.nombre, COUNT(*) AS compras
FROM CLIENTES, VENTAS
WHERE CLIENTES.codigo_cliente = VENTAS.codigo_cliente
GROUP BY CLIENTES.codigo_cliente, CLIENTES.nombre
ORDER BY compras DESC;
```

6) Promedio de unidades vendidas por producto, indicando el código del producto para el cliente 1.

```
SELECT ITEM_VENTAS.codigo_producto, AVG(ITEM_VENTAS.cantidad)
FROM ITEM_VENTAS, VENTAS
WHERE ITEM_VENTAS.numero_factura = VENTAS.numero_factura
  AND VENTAS.codigo_cliente = 1
GROUP BY ITEM_VENTAS.codigo_producto;
```